

PAYMENT TRANSACTION ANOMALY CLASSIFICATION

Overall Project Report

Domain: FinTech / Payment Security

Application: Payment Transaction Risk Intelligence Platform

Technology: Python • Pandas • NumPy • Scikit-learn • Plotly • Streamlit

A machine-learning based system for classifying payment transactions as routine or anomalous, supported by exploratory data analysis, model evaluation, interactive visualization, and a Streamlit web application.

Prepared as an academic project report

1. Abstract

Payment systems process a very large number of transactions, making manual inspection impractical. This project develops a machine-learning based Payment Transaction Anomaly Classification system that analyzes transaction characteristics and classifies records into routine and anomalous categories. The workflow covers data cleaning, exploratory data analysis, feature selection, classification, model evaluation, error analysis, and deployment through an interactive Streamlit application.

Three foundational classification approaches are considered: Logistic Regression, K-Nearest Neighbors (KNN), and Decision Tree. The application is designed as a Payment Transaction Risk Intelligence Platform with a command center, executive dashboard, transaction risk studio, transaction explorer, anomaly investigation, model intelligence, alert center, reports, and prediction history.

Important: Final numerical model metrics are intentionally not fabricated in this report. They should be inserted from the final executed evaluation results before submission.

2. Introduction

Digital payments generate transaction records containing information such as transaction type, amount, sender balances, receiver balances, and transaction sequence information. These variables can be analyzed to identify records whose patterns differ from routine transactions.

The objective is to create an end-to-end analytical system in which data is cleaned and explored, models are trained and evaluated, errors are examined, and the resulting model can be used through a user-friendly web interface.

3. Problem Definition

Develop a machine-learning system that classifies payment transaction records into two categories: routine and anomalous. The system should support data analysis, model comparison, evaluation using classification metrics, and interactive prediction through a web application.

3.1 Objectives

- Prepare and clean the payment transaction dataset.
- Understand transaction behaviour through exploratory data analysis.
- Select appropriate transaction features for classification.
- Train Logistic Regression, KNN, and Decision Tree models.
- Evaluate models using Accuracy, Precision, Recall, F1-score, Confusion Matrix, and ROC-AUC where appropriate.
- Perform error analysis, including false-positive and false-negative inspection.
- Save the selected final model for application use.
- Develop an interactive Streamlit application for transaction analysis.
- Allow users to enter transaction details and obtain a routine/anomalous classification from the trained model.

4. Dataset Description

The project uses a PaySim-style payment transaction dataset. The working data contains transaction, balance, and target information. The target variable is **isFraud**, where 0 represents a routine transaction and 1

represents an anomalous transaction for the project classification task.

Column	Role / Description
step	Transaction sequence/time-step information
type	Transaction type
amount	Transaction amount
nameOrig	Originator identifier; treated as an identifier rather than a model feature
oldbalanceOrig	Sender balance before transaction
newbalanceOrig	Sender balance after transaction
nameDest	Destination identifier; treated as an identifier rather than a model feature
oldbalanceDest	Receiver balance before transaction
newbalanceDest	Receiver balance after transaction
isFraud	Target: 0 = routine, 1 = anomalous
isFlaggedFraud	Existing transaction flag; excluded from the main feature set to avoid shortcut/leakage concerns

5. Data Cleaning and Preparation

The dataset was checked for missing values and duplicate records. The cleaning process removed records with a missing target value, filled missing receiver-balance values using their median values, filled the missing existing fraud-flag value using the mode, removed duplicate rows, reset the index, and saved the cleaned data as **payment_transactions_cleaned.csv**.

- Identify missing values column by column.
- Drop records where the target is missing.
- Fill missing oldbalanceDest and newbalanceDest values using their medians.
- Fill missing isFlaggedFraud using the mode.
- Remove duplicate rows.
- Reset the DataFrame index.
- Save the cleaned dataset.

6. Feature Selection and Preprocessing

Identifier columns nameOrig and nameDest are not used as predictive features. The primary model input consists of step, type, amount, oldbalanceOrig, newbalanceOrig, oldbalanceDest, and newbalanceDest.

Numeric variables are standardized using StandardScaler, while the categorical transaction type is encoded using OneHotEncoder with unknown categories ignored. The preprocessing and classifier are combined into scikit-learn Pipelines so that the same transformations are applied during training and prediction.

7. Exploratory Data Analysis

EDA is used to understand transaction distributions, class imbalance, transaction types, transaction amounts, balance behaviour, relationships among numeric variables, and potential outliers.

- Class distribution of routine versus anomalous transactions.
- Class percentages.

- Transaction type distribution.
- Transaction type versus anomaly classification.
- Anomaly rate by transaction type.
- Transaction amount distribution and amount versus anomaly.
- Sender balance analysis.
- Receiver balance analysis.
- Numeric correlation heatmap.
- Outlier analysis.

8. Machine Learning Methodology

The target variable is isFraud. The data is divided into training and testing subsets using an 80:20 split with stratification and a fixed random state for reproducibility.

8.1 Logistic Regression

Logistic Regression provides a baseline linear classification approach and is trained within the preprocessing pipeline.

8.2 K-Nearest Neighbors

KNN classifies a record based on nearby training examples. Standardization is important because distance calculations are affected by feature scale. During development, prediction probabilities on the complete test set were computationally expensive, so a stratified test sample was used for KNN evaluation.

8.3 Decision Tree

Decision Tree provides a non-linear, rule-based classification approach and can capture interactions among transaction characteristics.

9. Model Evaluation

The project evaluates classification performance using Accuracy, Precision, Recall, F1-score, and ROC-AUC. Confusion matrices and ROC curves are also generated.

Metric	Purpose
Accuracy	Overall proportion of correctly classified records.
Precision	Proportion of predicted anomalous records that are actually anomalous.
Recall	Proportion of actual anomalous records identified by the model.
F1-score	Harmonic mean of precision and recall.
ROC-AUC	Measures ranking/discrimination performance across classification thresholds.

Final results table — populate from the executed notebook before submission.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	—	—	—	—	—
KNN	—	—	—	—	—
Decision Tree	—	—	—	—	—

Comparison note: KNN was evaluated on a sampled test subset during development, while Logistic Regression and Decision Tree used the full test set. A common test subset should be used if a perfectly matched comparison is required.

10. Error Analysis

Error analysis is important because false positives and false negatives have different operational meanings. A false positive is a routine transaction classified as anomalous, while a false negative is an anomalous transaction classified as routine.

- Count false positives and false negatives.
- Inspect transaction amounts and transaction types associated with errors.
- Look for unusual balance changes in incorrectly classified records.
- Use findings to discuss model limitations and future improvements.

11. Streamlit Web Application

The project is designed as an interactive Streamlit application called the Payment Transaction Risk Intelligence Platform. The application connects the analytical workflow to a user-facing interface.

Module	Purpose
Command Center	High-level KPIs and transaction overview.
Executive Dashboard	Analytical charts and anomaly-rate analysis.
Transaction Risk Studio	Accept transaction inputs and classify them using the saved model.
Transaction Explorer	Search and filter transaction records.
Anomaly Investigation	Inspect anomalous records and amount distribution.
Explainable AI	Explore feature relationships and future model explanations.
Model Intelligence	Display model evaluation and comparison.
Alert Center	Display anomalous records requiring attention.
Reports	Download transaction data and summaries.
Prediction History	Track transactions analyzed during the current app session.
About System	Project, methodology, and technology information.

12. User Prediction Workflow

1. User opens Transaction Risk Studio.
2. User enters transaction type, amount, step, sender balances, and receiver balances.
3. The application creates a DataFrame with the same feature names used during model training.
4. The saved `final_model.pkl` pipeline applies the required preprocessing.
5. The classifier generates the prediction.
6. If probability output is available, the application displays the model's anomalous-class probability as a risk score.
7. The application displays either "Suspicious / Anomalous" or "Not Suspicious / Routine".
8. The transaction can be retained in the current session's prediction history.

13. System Architecture

Dataset → Cleaning → EDA → Feature Selection → Preprocessing Pipeline → Model Training → Model Evaluation → Final Model → Streamlit Application → User Input → Prediction → Result / Risk Score / History

The saved model should be a complete scikit-learn Pipeline containing the same preprocessing and classifier used during training. This helps prevent differences between training-time and application-time transformations.

14. Technology Stack

Technology	Use
Python	Core programming language
Pandas	Data loading, cleaning, transformation and analysis
NumPy	Numerical operations
Scikit-learn	Preprocessing, model training and evaluation
Plotly	Interactive visualization
Streamlit	Interactive web application
Joblib	Saving and loading the trained model
GitHub	Version control and project source hosting
Streamlit Community Cloud	Planned application deployment

15. Testing and Validation Plan

- Verify that the cleaned dataset loads successfully.
- Verify that required model features are present.
- Verify that final_model.pkl can be loaded without errors.
- Test valid transaction inputs.
- Test zero and very small transaction amounts.
- Test large transaction amounts.
- Test each supported transaction type.
- Test invalid values using Streamlit validation.
- Verify prediction output is displayed clearly.
- Verify the model uses the same preprocessing as training.
- Verify downloaded reports contain expected records and columns.

16. Limitations

- The project depends on the quality and representativeness of the dataset.
- Synthetic or simulated payment data may not represent every real-world payment behaviour.
- The classification target depends on the dataset's labeling methodology.
- A high classification score does not by itself prove that a real-world transaction is fraudulent.

- KNN evaluation during development used a stratified test sample because full-test probability prediction was computationally expensive.
- The application risk score is a model output and not a guarantee of fraud.
- Current prediction history is session-based unless persistent storage is added.

17. Future Scope

- Add SHAP-based local and global explanations.
- Add persistent storage for prediction history and investigation records.
- Add role-based access for analysts and administrators.
- Add configurable anomaly thresholds.
- Add batch CSV prediction and downloadable scored results.
- Add automated alert and notification workflows.
- Experiment with class-imbalance handling and threshold tuning.
- Evaluate additional models and hyperparameter tuning.
- Add data-drift and model-performance monitoring.
- Integrate with secure production payment systems only after appropriate validation, privacy, and security controls.

18. Conclusion

The Payment Transaction Anomaly Classification project provides an end-to-end machine-learning workflow for analyzing payment transaction records. It combines data preparation, exploratory analysis, classification models, evaluation, error analysis, and a Streamlit-based user interface.

The resulting Payment Transaction Risk Intelligence Platform is intended to make the analytical workflow accessible through a practical interface: users can review transaction patterns, explore anomalous records, compare model performance, and enter transaction characteristics for classification using the trained model.

Before final academic submission, the actual executed model metrics, final model selection, final EDA figures, dataset citation/license information, application screenshots, and deployment details should be inserted so every reported result is traceable to the final project files.

19. References / Project Resources

- Scikit-learn documentation — preprocessing, pipelines, classification models, and evaluation metrics.
- Streamlit documentation — application development and deployment.
- Pandas documentation — data manipulation and analysis.
- Plotly documentation — interactive visualization.
- The selected PaySim-style transaction dataset and its original source/citation should be recorded in the final submission.

Submission note: This is an overall project report. The final academic version should include actual plots, numerical evaluation results, selected final model, dataset source/license, screenshots of the completed web application, and deployment URL after those components are finalized.